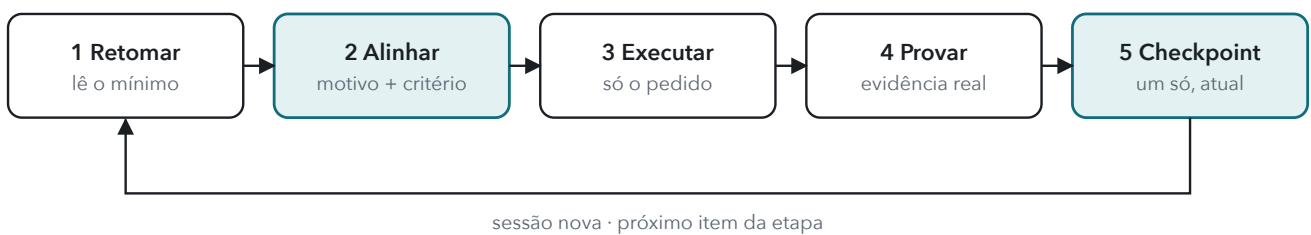


auxcode

Cada mudança ligada ao objetivo do projeto. Nada dado por pronto sem evidência. Retomada lendo o mínimo.



Uma disciplina que o modelo aplica em pontos fixos do trabalho

auxcode é um conjunto de instruções que o Claude Code carrega ao trabalhar num aplicativo. A partir daí o modelo exige de si mesmo três coisas: um motivo antes de codar, uma prova antes de dizer pronto e leitura mínima ao retomar.

1. Nada sem motivo

Antes de tocar em código, o Claude escreve uma linha ligando o pedido a um item do roadmap, a uma regra do projeto ou a um defeito comprovado. Sem isso, o pedido vai para o backlog e vira pergunta, não código.

2. Nada pronto sem evidência

"Pronto" é o critério observável da linha cumprido no fluxo real, relatado em três estados: feito, verificado, publicado. O que não foi verificado é dito, não escondido.

3. Retomar lendo o mínimo

Um mapa de fontes e um checkpoint curto vivem no próprio projeto. A sessão nova lê os dois, confere o estado real e segue, sem reler a documentação inteira.

Para quem

Para quem constrói aplicativos com o Claude Code, sozinho ou em equipe pequena, tem um roadmap ou quer passar a ter, e já viu uma sessão fugir do plano: refatoração que ninguém pediu, "pronto" que não estava, meia hora relendo documentos antes de escrever a primeira linha.

O que ela não é

Não é um monitor automático nem um gerador de relatórios. Não mede tokens nem percentuais de contexto. É uma disciplina de decisão com saídas que você consegue conferir: uma linha no chat antes do código, um relato em três estados no fim e um checkpoint gravado no projeto.

Neste documento

O problema que ela resolve	3	Revisar um aplicativo existente	8
O ciclo em cinco passos	4	Provar que está pronto	9
A linha de alinhamento	5	Três portas de entrada	10
Retomar lendo o mínimo	6	Como funciona por dentro	11
O checkpoint	7	Instalar, usar e limites	12

Cinco falhas que nascem sempre nos mesmos lugares

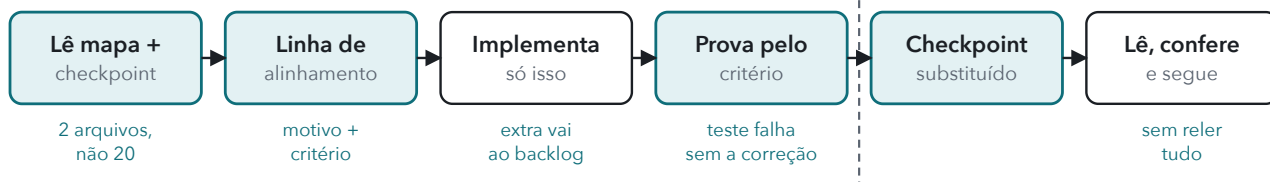
Sessões longas com um agente de código repetem um padrão. Nenhuma dessas falhas é falta de capacidade do modelo. Todas são falta de disciplina em pontos previsíveis do trabalho.

- **Desvio silencioso.** O pedido era uma correção; a sessão termina com uma refatoração, uma biblioteca nova e um tema escuro que ninguém pediu.
- **"Pronto" que não está.** O código compila, o modelo leu o próprio diff e declarou sucesso. O fluxo nunca rodou.
- **Releitura a cada sessão.** README, roadmap inteiro, todas as decisões, histórico. O contexto enche antes de a primeira linha ser escrita.
- **Tentativa repetida.** O que foi descartado ontem, com motivo, é tentado de novo hoje porque o motivo se perdeu.
- **Roadmap como enfeite.** O plano existe, mas nada liga o que está sendo feito ao item que deveria estar sendo fechado.

Sem disciplina



Com a skill

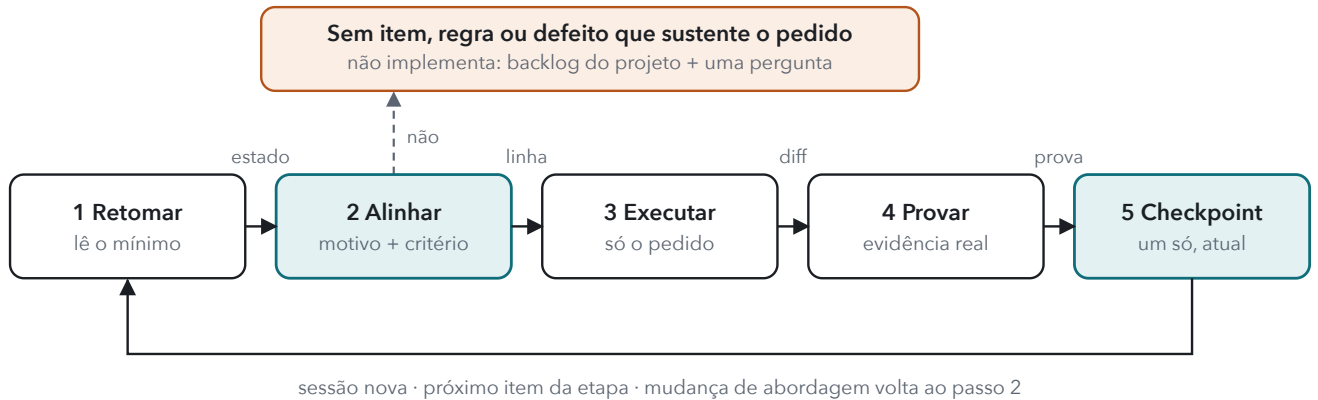


O mesmo pedido em duas sessões. Em cima, o padrão sem disciplina; embaixo, os pontos fixos que a skill impõe. Os blocos em destaque são os instrumentos da skill. Depois da linha tracejada, a sessão seguinte.

A skill não tenta impedir cada erro possível. Ela coloca uma checagem barata onde os erros costumam nascer: antes de codar, antes de dizer pronto e ao retomar.

O ciclo em cinco passos

Tudo o que a skill faz cabe num ciclo. Ele roda uma vez por pedido e recomeça na sessão seguinte ou no próximo item da etapa. Quando a abordagem muda ou aparece trabalho extra no meio do caminho, o modelo volta ao passo 2 e refaz a linha.



O ciclo completo. Os passos em destaque são os dois instrumentos: a linha de alinhamento e o checkpoint. O desvio tracejado é o que impede código sem motivo.

1 Retomar	2 Alinhar	3 Executar	4 Provar	5 Checkpoint
Lê o mapa de fontes, o checkpoint e só as fontes da tarefa ativa. Do roadmap, a etapa atual e a seguinte. Confere o checkpoint contra branch, testes e deploy.	Escreve a linha de alinhamento no chat. Sem sustentação, não implementa: anota no backlog e faz a menor pergunta que decide.	Só o que a linha cobre, mais dependências técnicas. Melhoria não pedida vira uma linha no backlog. Defeito só com esperado e observado registrados.	Valida pelo critério da linha, no fluxo afetado. Revisa o diff contra o pedido. Relata feito, verificado, publicado e o que ficou sem verificar.	Substitui o anterior: tarefa, estado, próximo passo, decisões, descartados, fora do escopo. O próximo passo é o próximo item da etapa.

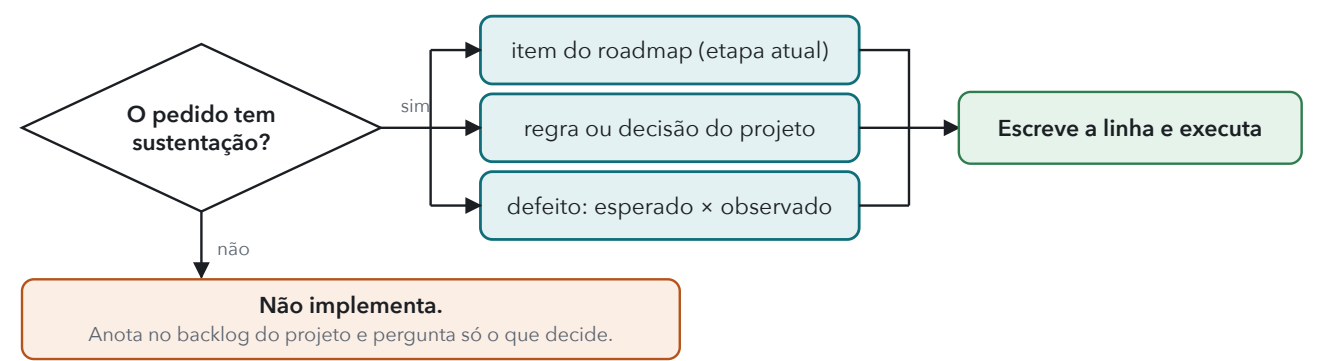
Nada disso exige reler o roadmap inteiro a cada comando, emitir relatórios cerimoniais ou pedir aprovação repetida. A linha é uma frase; o relato, três palavras e uma lista curta; o checkpoint, meia tela.

A linha de alinhamento

É uma frase com forma fixa, escrita no chat antes de qualquer código. Ela obriga o modelo a tornar explícita a ligação entre o que vai fazer e o motivo, e define desde o início o que vai contar como "pronto". Como a forma é sempre a mesma, você confere em dois segundos.



As cinco partes da linha. As duas em destaque são as que fazem o trabalho: a sustentação e o critério.



A decisão que a linha força. Um "não" aqui é o que separa um auxiliar disciplinado de um gerador de código.

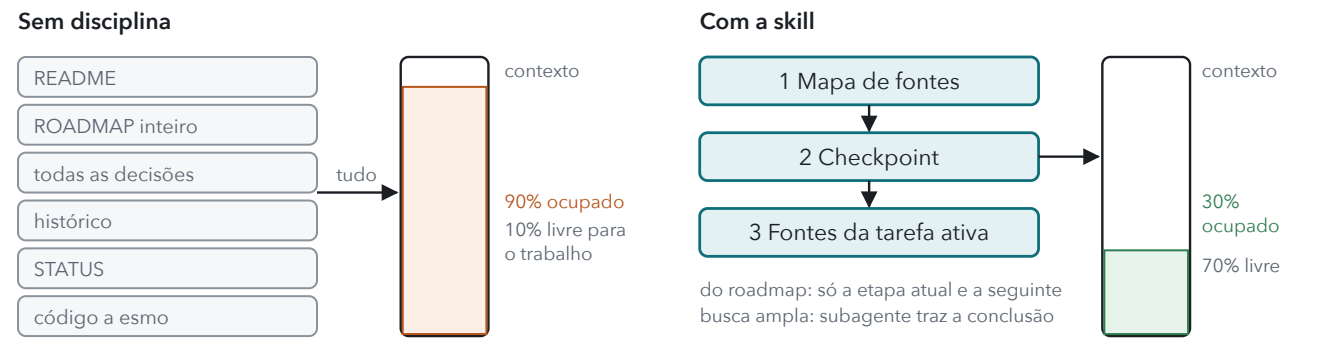
Três exemplos

- Alinhamento: `paginar a lista de pedidos` `atende` `Etapa 2, item 3, em docs/ROADMAP.md`.
Pronto quando: `1.000 pedidos carregam em páginas de 50` e o teste de paginação passa.
- Alinhamento: `corrigir o total com desconto` `atende a regra "total nunca negativo"` em `docs/adr/ADR-009.md`; `defeito observado no teste test_total_desconto`.
Pronto quando: o teste falha sem a correção e passa com ela.
- Sem item, regra ou defeito que sustente `"adicionar tema escuro"`. Anotado em `docs/ROADMAP.md §Backlog`. Quer priorizar agora ou seguimos na Etapa 2?

Critério observável é o que dá para ver ou medir: um teste, um número, um comportamento na tela. "Funcionar bem" e "ficar melhor" não servem.

Retomar lendo o mínimo

O maior desperdício de uma sessão nova é reconstruir o contexto do zero. A skill troca isso por uma ordem de leitura fixa e curta: primeiro o mapa de fontes, depois o checkpoint, depois só as fontes que o checkpoint aponta para a tarefa ativa. Do roadmap, apenas a etapa atual e a seguinte. Busca ampla em muitos arquivos vai para um subagente, que devolve só a conclusão.



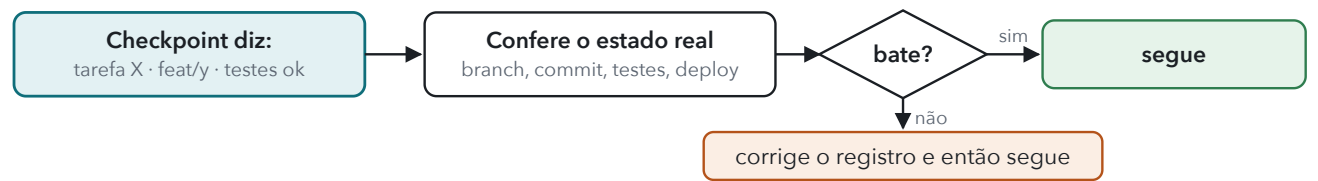
Proporções ilustrativas. O ponto não é o número, é a ordem: mapa, checkpoint, fontes da tarefa, e parar.

O mapa de fontes

Uma tabela de quatro linhas que quase não muda: só caminho, seção e papel. Ela responde "onde está o quê" para que o modelo não precise descobrir isso de novo a cada sessão.

PAPEL	ONDE	NOTA
Propósito e público	docs/produto.md §1	
Regras e decisões	docs/adr/ , índice em docs/adr/README.md	ADR-012 substitui ADR-004
Roadmap e etapa atual	docs/ROADMAP.md §Etapa 2	
Estado e evidências	docs/STATUS.md , CI, ambiente de preview	

Checkpoint é atalho, não prova



Um registro antigo pode estar errado. Data recente não é garantia; o estado real dos arquivos é.

O checkpoint

Meia tela de texto, gravada no arquivo de estado que o projeto já usa, substituída a cada marco. Registra só o que muda a próxima decisão. Não é diário: o anterior some, e o que merece durar vai para o documento certo.

Checkpoint – 2026-09-06 19:40 · feat/paginacao @ a1b2c3d

Etapa: 2 (docs/ROADMAP.md \$Etapa 2)

Tarefa: paginar a lista de pedidos

Pronto quando: 1.000 pedidos em páginas de 50; teste de paginação passa

Estado: feito: endpoint e UI · verificado: teste e fluxo manual · publicado: não

Próximo passo: deploy em preview e conferir

Decisões desta rodada: cursor em vez de offset (ADR-014; substitui a nota da ADR-010)

Fatos verificados: 1.000 registros em 2,1 s

Hipóteses abertas: índice em created_at pode faltar

Descartado: paginação no cliente · lenta com 5.000

Fora do escopo, anotado em: ROADMAP \$Backlog (filtro por período)

Não verificado: tema escuro · sem ferramenta visual

Cabeçalho. Data, branch e commit. É o que a retomada confere contra o estado real; sem isso o checkpoint não vale como atalho.

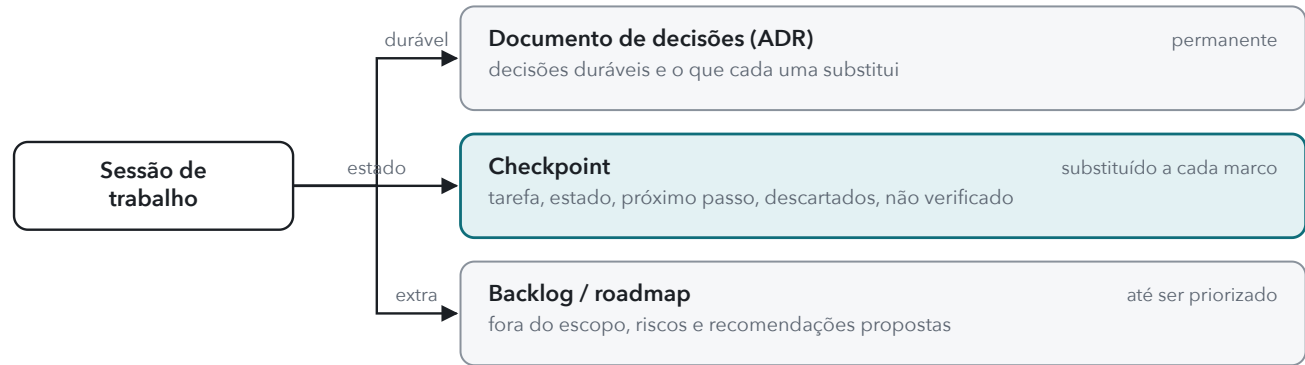
Etapa, tarefa, pronto quando. A linha de alinhamento, guardada. Quem retoma sabe o que estava sendo feito e o que conta como concluído.

Estado em três níveis. Feito, verificado, publicado. "Publicado: não" evita o "achei que já estava no ar".

Decisões e o que substituem. Por referência. A decisão em si vive no documento de decisões.

Fatos, hipóteses, descartados. Separados de propósito. Um descartado com motivo é o que impede a tentativa repetida.

Fora do escopo e não verificado. O que foi visto e não feito, e o que não pôde ser provado. Nada some em silêncio.



Três destinos, três tempos de vida. O checkpoint só referencia decisões; quem as guarda é o documento de decisões. É isso que permite substituí-lo sem perder nada.

Regras do registro: omitir campo vazio; nada de logs, código ou segredo; atualizar em marcos, mudança de direção e encerramento, não a cada ação. Se não puder gravar arquivos, o modelo entrega o registro no chat e avisa que a retomada depende dele.

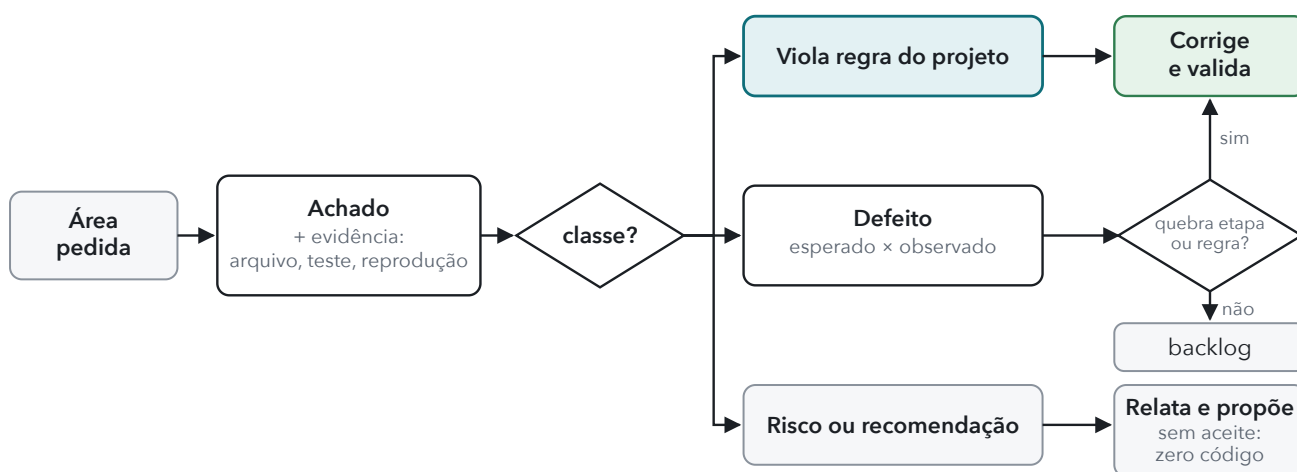
Revisar e adequar sem inventar regras

Revisar um aplicativo é onde um agente mais tenta "melhorar" o que ninguém pediu. A skill resolve isso com duas regras: uma base de comparação com ordem definida e uma classe obrigatória para cada achado, que decide o que acontece com ele.

Base de comparação, nesta ordem

1. **Regras e decisões do projeto.** O que está escrito e foi aceito.
2. **Diretrizes que o projeto adotou** de uma biblioteca, como o Guia de aplicativos.
3. **Padrões que o próprio código já pratica.**

Norma genérica ou preferência do modelo não entra na lista. Regra praticada mas não escrita é proposta de regra, não desvio.



Cada achado sai com evidência e uma classe. Só duas classes viram código, e uma delas ainda passa por um filtro. A terceira vira proposta.

Pedido de adequação

Corrige e valida cada item, priorizando por objetivo, etapa e dependência. Não termina só com recomendações.

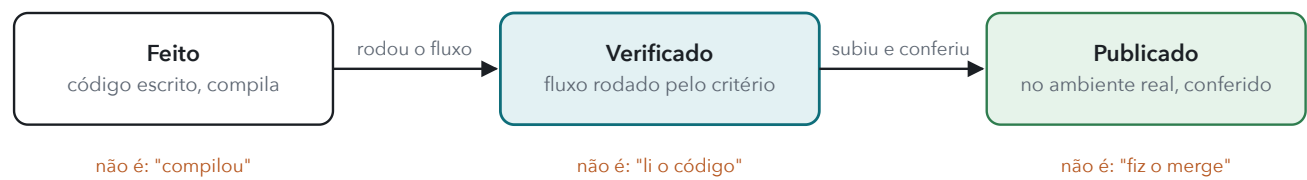
Pedido só de diagnóstico

Entrega os achados com evidência e classe. Não altera a implementação, nem "só um ajuste".

A inspeção não cresce além da área pedida. O que o modelo viu fora dela vira uma linha no backlog, não uma auditoria do sistema inteiro.

Provar que está pronto

"Pronto" tem três significados diferentes e a conversa costuma misturá-los. A skill obriga o relato a separá-los e a dizer, junto, o que ficou sem verificar e por quê.



Os três estados e o falso amigo de cada um. Compilar não prova; inspecionar não é executar; merge não é publicação.

Três regras contra erros que passam despercebidos

Teste novo tem que falhar sem a correção

	FALHA	PASSA
sem a correção	esperado	não prova nada
com a correção	correção errada	esperado

Um teste escrito junto com a correção costuma passar nos dois casos. Verde assim é verde por motivo errado.

Ausência exige contagem, não amostra

```
$ tail -12 app.log
... nada de errado ...

$ grep -c ERRO
app.log
37
```

"Não há X" só vale com a contagem completa. Saída cortada é evidência cortada. Agregar com contagem ou filtro também poupa contexto.

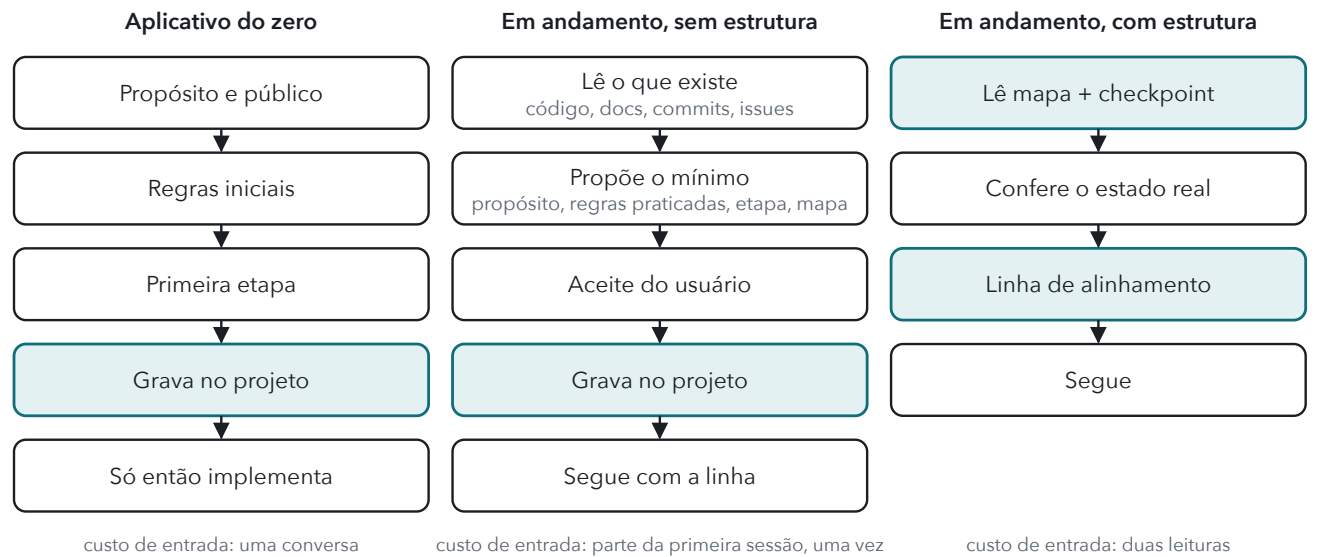
Diga o que ficou sem verificar

Interface sem ferramenta visual, ambiente sem acesso, dado de produção indisponível: acontece. O relato nomeia o que não foi provado e por quê, em vez de declarar sucesso integral. Evidência só vale no ambiente que o projeto exige; rodou em outro runtime, diga qual e não conte como verificado. Antes disso, o diff é revisto contra o pedido e acréscimos sem justificativa são removidos.

No roadmap, um item só é marcado como concluído com evidência. Constar no plano, ou até constar como feito no plano, não é conclusão.

Três portas de entrada

A skill serve para um aplicativo novo e para um que já existe, com ou sem documentação organizada. Ela não exige reorganizar nada: usa o arquivo de estado que o projeto já tem e só cria um quando não há equivalente. O que muda em cada caso é o que acontece antes da primeira linha de alinhamento.



As três portas convergem no mesmo ponto: um projeto com propósito, regras, etapa e lugar de estado, e a partir daí o ciclo de cinco passos.

Do zero

Nada de funcionalidades antes de propósito, público, regras iniciais e primeira etapa. Propostas do modelo são propostas até serem aceitas; ele não inventa público, prazo nem prioridade.

Sem estrutura

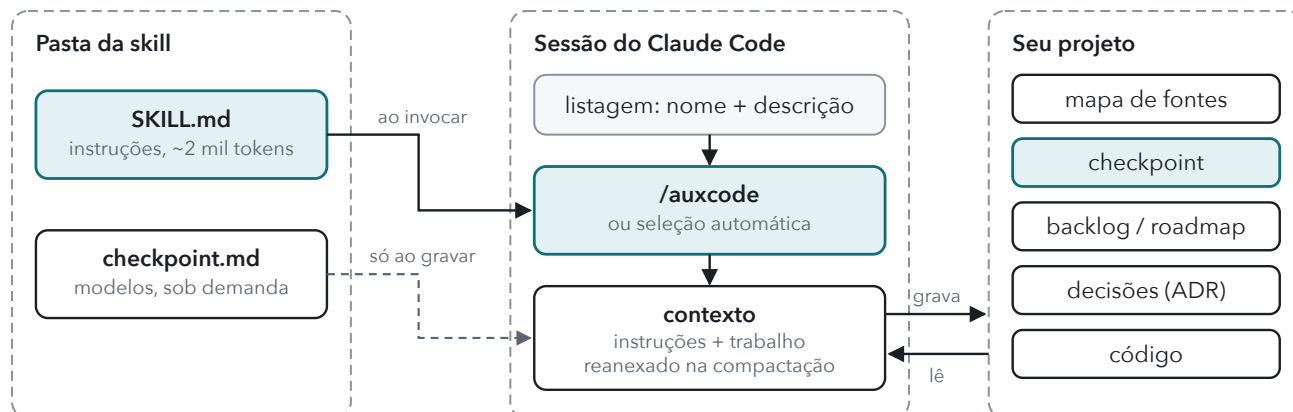
O projeto não trava nem é obrigado a se reorganizar. A proposta mínima é curta: uma linha de propósito, as regras que o código já pratica, a etapa atual e a seguinte, o mapa. Depois do aceite, o projeto passa a ter estrutura.

Com estrutura

O caminho rápido. Se o projeto segue o Guia de aplicativos, o documento de parâmetros é a fonte de propósito e regras que a skill procura primeiro.

Como funciona por dentro

Uma skill do Claude Code é uma pasta com um arquivo de instruções. Não há código rodando, monitor ou plugin: quando a skill é invocada, o texto entra na conversa e o modelo passa a segui-lo. Tudo o que precisa durar além da sessão fica em arquivos do seu projeto.



O que entra na sessão e o que fica no projeto. A seta tracejada é a leitura sob demanda: os modelos não custam contexto até serem necessários.

Invocação

Por comando, `/auxcode <pedido>`, ou por seleção automática: a descrição da skill diz quando ela se aplica, e frases como "retoma", "o que falta para fechar a etapa" ou "revise este app" acionam o carregamento sem comando.

O que custa contexto

O arquivo de instruções, cerca de 2 mil tokens, uma vez por invocação. Os modelos de checkpoint só quando o modelo vai gravar. A economia real vem do comportamento: ler menos, não repetir tentativas e não escrever código fora do escopo.

Sessões longas

Quando a conversa é compactada, o Claude Code reanexa a invocação mais recente de cada skill, até 5.000 tokens por skill e 25.000 no total. Esta skill cabe inteira. O que não sobrevive à compactação é o raciocínio da sessão; por isso o estado fica no checkpoint em disco, não na memória do modelo.

O que ela não toca

A skill não edita `CLAUDE.md`, configurações nem hooks para se instalar. Quem decide integrá-la ao `CLAUDE.md` de um projeto é você, com uma referência de três linhas.

Instalar, usar e limites

Instalação

Clone o repositório direto no destino. Só `SKILL.md` e `checkpoint.md` entram em uso; o resto é documentação. Se a pasta de skills não existia quando a sessão começou, reinicie o Claude Code. Para atualizar, `git pull` na mesma pasta.

USO	DESTINO
Só num projeto	<code><projeto>/.claude/skills/auxcode/</code>
Pessoal, em todos	<code>~/.claude/skills/auxcode/</code>

```
git clone
https://github.com/pnetopsx/auxcode.git
~/.claude/skills/auxcode
```

Quatro pedidos típicos

```
/auxcode Quero começar um app de agenda.
Defina comigo propósito, regras e a
primeira etapa antes de escolher
funcionalidades.
```

```
/auxcode revisar o fluxo de cadastro
contra as regras deste projeto. Corrija os
desvios comprovados e valide o fluxo.
```

```
/auxcode Verifique o cálculo do
fechamento. Só diagnóstico, com
evidências; não altere arquivos.
```

```
/auxcode retomar
```

O que você vai ver

1. Antes de qualquer código, a linha `Alinhamento: ...`. Se ela não fechar, o pedido vai ao backlog e vira pergunta.
2. Numa revisão, achados com evidência e classe; correções validadas uma a uma quando o pedido foi adequar.
3. No fim, o relato em feito, verificado e publicado, o que ficou sem verificar, e o checkpoint substituído no arquivo de estado do projeto.
4. Na retomada, leitura de mapa, checkpoint e fontes da tarefa, com o checkpoint conferido contra o estado real.

Integração opcional

No `CLAUDE.md` do aplicativo, três linhas bastam para lembrar a skill e apontar o arquivo de estado:

```
Neste aplicativo, aplique a skill auxcode.
Estado de trabalho (mapa e checkpoint):
docs/STATUS.md. Na retomada, leia-o
primeiro e confira-o contra o estado real
antes de continuar.
```

Limites e estado atual

A skill orienta decisões e registros. Não é um monitor automático, não mede tokens nem percentuais de contexto e não garante ausência de erro ou cumprimento de prazo. Depende de o projeto ter, ou passar a ter, um roadmap e um lugar de estado.

Versão 0.3.1, setembro de 2026. Estrutura validada e duas primeiras sessões de uso real feitas, retomada e revisão; os ajustes desta versão vieram delas. O que você observar no uso é a próxima evidência: relate.

Repositório: github.com/pnetopsx/auxcode, licença MIT. Documentação oficial de skills: code.claude.com/docs/en/skills.